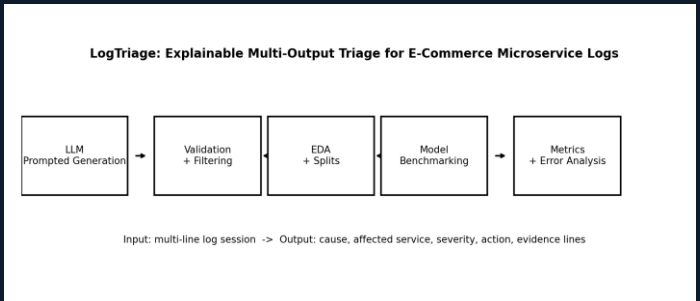
LogTriage

Explainable Multi-Output Triage for E-Commerce Microservice Logs

NLP Course Final Presentation • Synthetic benchmark • Real local baseline evaluation

Reproduce results: python src/train_baselines.py

Review and refined project definition



Motivating use case

Checkout incidents create logs across gateway, auth, cart, payment, inventory, orders, and queues.
Operators need fast, structured triage: what failed, where, how severe, what action, and what evidence.
Manual reading and keyword search are slow and inconsistent.

Input

```
[1] gateway INFO POST /checkout request_id=req_912
[2] checkout INFO validating cart_id=cart_771
[3] payment-service WARN provider_latency=4900ms
[4] payment-service ERROR payment gateway timeout
after 5000ms
[5] order-service ERROR order creation failed: auth
missing
[6] gateway ERROR status=503 route=/checkout
```

Output

```
{
  "failure_cause": "payment_gateway_timeout",
  "affected_service": "payment-service",
  "severity": "high",
  "recommended_action":
"check_payment_gateway_and_retry",
  "evidence_lines": [3, 4, 6]
}
```

Formal ML framing

Multi-output NLP classification + evidence-line selection from numbered log sessions.
Fixed schema enables measurable outputs: cause, service, severity, action, evidence lines.
Compares rule-based, TF-IDF/LogReg, prompt-based LLM baselines, and prepared transformer training pipeline.
Metrics: macro-F1/accuracy per field, evidence-line F1, and full JSON exact match.

Project achievements and novelty

Completed project artifacts

5,000
synthetic log sessions

3 splits
train / validation / test

Fixed
cause, service, severity, action taxonomy

0
schema validation issues

6 Tested Baselines
majority, rules, TF-IDF LogReg/SVM/NB, Char-SVM

EDA + metrics
charts, confusion matrix, error analysis

Novelty claim

Not only anomaly detection: the system returns operator-ready triage decisions.
Predicts cause + affected service + severity + action + supporting evidence lines.
Synthetic benchmark is explainable and measurable end-to-end.

Anomaly detection vs. LogTriage

Traditional

Normal / abnormal
Limited operator context

→

LogTriage

Cause + service + severity
Action + evidence

Fixed label taxonomy

Controlled outputs enable fair, measurable evaluation.

Label type	Taxonomy size	In dataset	Examples
Failure cause	10	10	payment_gateway_timeout, database_query_failure, message_queue_backlog, ...
Affected service	12	7	payment-service, queue-service, database-service, gateway-service, ...
Severity	4	4	low, medium, high, critical
Recommended action	10	10	scale_queue_consumers, rollback_deployment, check_database_query, ...
Evidence lines	1–8 lines	avg 3.3	1-based log line indices supporting cause and action

Cause and action are paired in the generator; severity is intentionally harder because it depends on impact context.

Related work: gaps vs. LogTriage

Existing datasets are strong RCA/anomaly references; the gap is operator-ready text triage with action + evidence supervision.

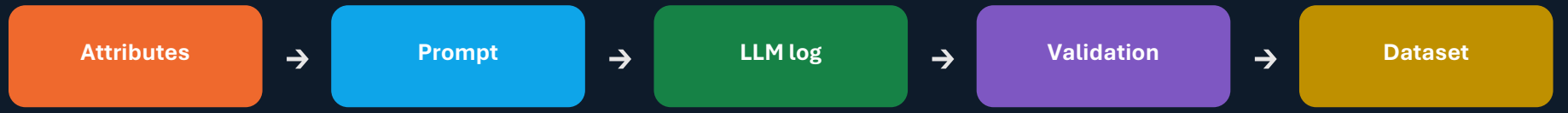
Careful novelty claim: RCA/log anomaly detection is not new. LogTriage adds a controlled synthetic NLP benchmark for cause + service + severity + action + evidence lines.

Project / dataset	Main focus	What it gives	Gap vs. LogTriage
RCAEval	Microservice RCA	735 injected failure cases from Online Boutique, Sock Shop and Train Ticket; includes metrics, logs, traces and annotated root-cause service / indicator.	Strong RCA benchmark, but output stops mainly at root-cause localization. It does not provide fixed action labels, severity, or line-level evidence as operator-ready JSON.
LO2	API anomaly / failure prediction	Production microservice logs + metrics connected to API errors; useful for realistic failure/anomaly patterns and API-level symptoms.	Less focused on what the operator should do next. It lacks a fixed recommended-action taxonomy and traceable evidence-line supervision.
AnoMod	Anomaly + RCA	Multi-modal microservice dataset with logs, metrics, traces, API responses and code coverage; covers performance, service, DB and code-level faults.	Broader and multi-modal. LogTriage is narrower: text-log NLP triage for e-commerce sessions with cause + service + severity + action + evidence.
GAIA / AIOps	AIOps anomaly / fault localization	Open AIOps data for anomaly detection, log analysis and fault localization; useful for operational patterns and evaluation inspiration.	General AIOps source data, not a controlled benchmark for operator decision JSON or action/evidence labels.
LogHub	Real system logs	Large public collection of real logs from many systems; useful for realistic log wording, parsing, templates and anomaly baselines.	Usually lacks rich incident labels: root cause, severity, recommended action and exact supporting evidence lines.
LogSage / agents	LLM RCA + remediation	LLM-based diagnostic/remediation framework over raw logs; conceptually close because it connects diagnosis with possible resolution.	Different domain and agent framework. LogTriage is a supervised, measurable NLP benchmark with fixed taxonomies and synthetic controlled data.

Takeaway: existing work gives realistic RCA/anomaly patterns; LogTriage fills the gap by turning text logs into measurable operator triage: cause + service + severity + action + evidence lines.

Synthetic log generation: uniqueness and quality controls

Main challenge: generate logs that are realistic, diverse, and challenging without leaking labels.



1. Avoid label leakage

Do not include exact class names as obvious text
Use paraphrases and indirect symptoms
Check shortcut keywords and repeated label phrases

2. Preserve realism

Each line must look like a real log entry
Session must follow a plausible timeline
Use service interactions inspired by LogHub/RCA datasets

3. Enforce diversity

Vary service path, length, style, severity and noise
Deduplicate and check similarity
Generate more than small template variants

Noise-level checks

Low/medium/high noise is validated using log length, evidence-line ratio, unrelated-service count, and distractor WARN/ERROR lines.
Noise distribution is validated in EDA; per-noise model metrics are reported on a dedicated slide (TF-IDF SVM).

Before training: JSON/schema validation, allowed labels, label coverage, cause-action consistency, evidence-line validity, duplicate/similarity checks, manual review sample.

Review methodology

Controlled attributes → **LLM synthetic generation** → Schema + label validation → EDA + split → Baselines + evaluation

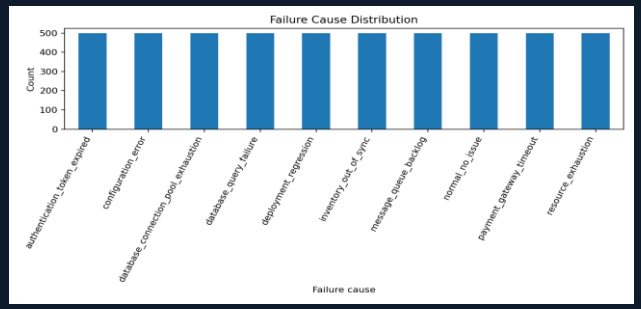
Synthetic data recipe

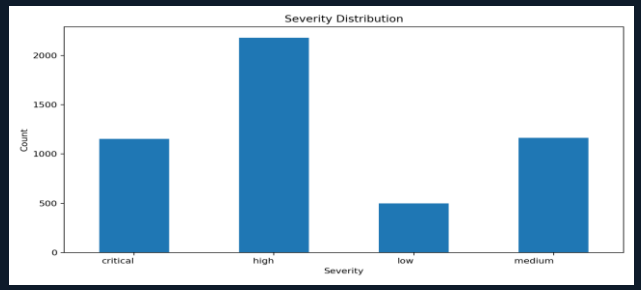
LLM prompted with fixed attributes: cause, service, severity, action, noise, log length, and style.
JSON schema forces numbered logs and evidence-line labels.
Validation removes malformed JSON, invalid labels, invalid line references, and duplicates.

Dataset

5,000 examples
70 / 15 / 15 split (3500 / 750 / 750)
Avg. 7.67 log lines

EDA checks





Model experiments

Rule-based baseline	evaluated
TF-IDF + Logistic Reg.	evaluated
TF-IDF + Linear SVM	evaluated
TF-IDF + Naive Bayes	evaluated
Character TF-IDF + SVM	evaluated
Zero/few-shot LLM	evaluated (n=30)
DistilBERT classifier	evaluated (n=75)

Example triage: rules vs. TF-IDF SVM

Test session sess_02876 — keyword rules miss affected service; ML learns log cues.

Log excerpt

```
[1] 2026-05-01 10:14:01 shipping-service INFO feature_flag
fast_inventory=enabled
```

[2] 2026-05-01 10:14:02 notification-service INFO metrics cpu=54% mem=1438MB
[3] 2026-05-01 10:14:03 payment-service INFO heartbeat ok instance=2
[4] 2026-05-01 10:14:04 order-service INFO publishing order_created event order_id=7046
[5] 2026-05-01 10:14:05 queue-service WARN topic=order-events lag=9363 consumers=2
[6] 2026-05-01 10:14:06 queue-service ERROR message queue backlog exceeded threshold

Gold
cause: message_queue_backlog
service: queue-service
severity: high
action: scale_queue_consumers

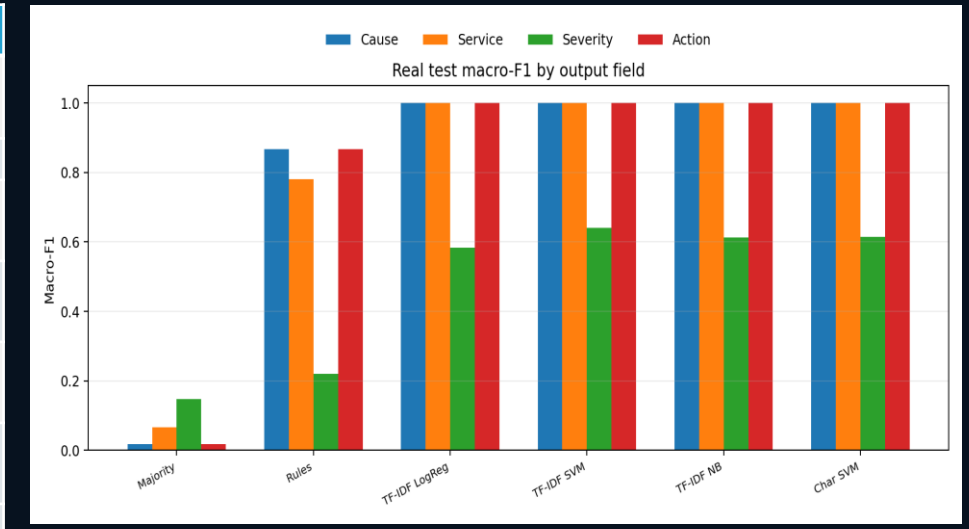
Rules
cause: message_queue_backlog
service: notification-service
severity: medium
action: scale_queue_consumers

TF-IDF SVM
cause: message_queue_backlog
service: queue-service
severity: medium
action: scale_queue_consumers

- Rules match cause/action keywords but pick notification-service instead of queue-service.
- TF-IDF SVM recovers the correct service from queue-related log lines.
- Severity remains hard for both models on this example (gold=high, pred=medium).

Results: real model testing on held-out test set

Model	Cause F1	Service F1	Severity F1	Action F1	Full exact	Evidence F1
Majority	0.018	0.066	0.148	0.018	0	0.822
Rules	0.804	0.253	0.481	0.804	0.175	0.861
TF-IDF LogReg	1	1	0.584	1	0.544	0.886
TF-IDF SVM	1	1	0.649	1	0.572	0.886
TF-IDF NB	1	0.997	0.421	1	0.492	0.886
Char SVM	1	1	0.609	1	0.552	0.886
LLM zero-shot	1	0.734	0.441	1	0.3	0.942
LLM few-shot	1	1	0.643	1	0.533	0.99
DistilRoBERTa	0.788	0.176	0.129	0.75	0.08	0.873



Best cause/service/action

TF-IDF models reach ~1.000 F1 on cause and action; service is 1.000 for most models (0.997 for NB) because the synthetic data has strong learnable signals.

Hardest output

Severity is lower: it depends on impact/context, not only words. Best real severity F1: 0.649 with Linear SVM.

Strict exact match

Full exact requires all 4 labels correct together; it is limited mainly by severity errors.

LLM subset scope

LLM rows above are real API runs on n=30 (not the full 750). Valid JSON rate: 100% for both modes. DistilRoBERTa fine-tuned on n=300 train / n=75 test (CPU subset).

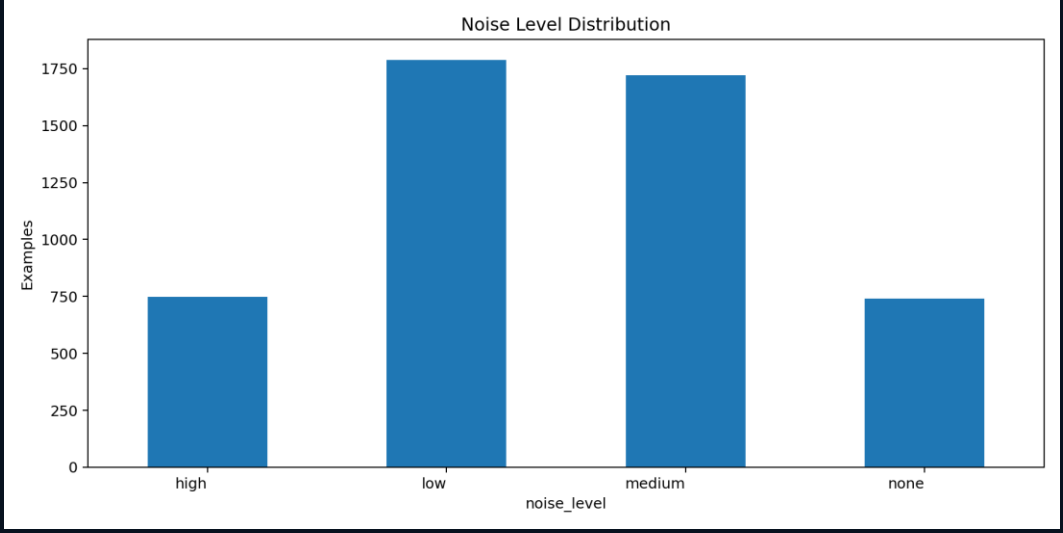
Note

Best local baseline (full n=750): TF-IDF SVM — 0.572 full exact. Best LLM (n=30): few-shot — 0.533. DistilRoBERTa (n=75): 0.080 full exact.

Robustness by noise level (TF-IDF SVM)

Best model performance on the same held-out test split, grouped by low / medium / high noise.

Noise level	N	Severity F1	Full exact
low	195	0.650	0.569
medium	269	0.657	0.584
high	203	0.645	0.571

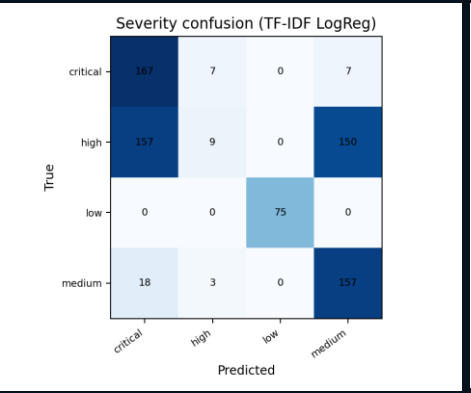


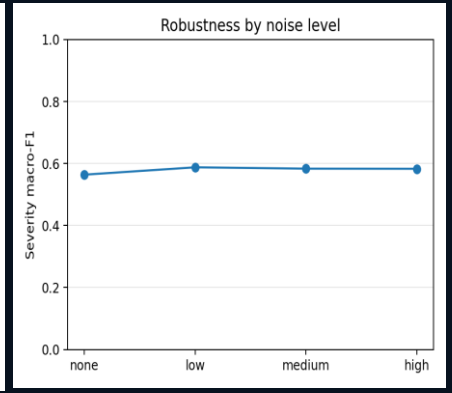
Interpretation

- Cause, service, and action stay at 1.000 macro-F1 across all noise levels.
- Severity F1 is stable (~0.645–0.657) but remains the bottleneck.
- Full exact match stays near 57% — severity errors dominate end-to-end success.

Detailed results: label accuracies and evidence metrics

Model	Cause Acc	Service Acc	Severity Acc	Action Acc	Evidence P/R/F1
Majority	0.1	0.3	0.421	0.1	0.767/0.886/0.822
Rules	0.825	0.412	0.453	0.825	0.795/0.941/0.861
TF-IDF LogReg	1	1	0.544	1	0.838/0.940/0.886
TF-IDF SVM	1	1	0.572	1	0.838/0.940/0.886
TF-IDF NB	1	0.997	0.492	1	0.838/0.940/0.886
Char SVM	1	1	0.552	1	0.838/0.940/0.886
LLM zero-shot	1	0.8	0.367	1	0.989/0.900/0.942
LLM few-shot	1	1	0.533	1	0.990/0.990/0.990
DistilRoBERTa	0.787	0.44	0.347	0.747	0.813/0.943/0.873





What the real test shows

Cause/service/action are almost perfectly learnable by TF-IDF, which is useful but also signals possible lexical shortcuts in the synthetic data.

Severity confusion

Most mistakes are high/critical/medium shifts. Low severity is easy; impact level needs better generation rules and harder examples.

Evidence-line metric

Evidence F1 compares selected log line numbers vs. gold evidence lines. The shared heuristic reaches ~0.86–0.89 F1; future work should train a dedicated line-level evidence model.

Next data fix

Generate less label-leaking logs, more paraphrases, and high-noise cases with misleading symptoms.

Error analysis categories

Patterns from real test-set predictions across all six baselines.

1. Severity confusion (all models)

- Most full-exact failures come from high / critical / medium mislabels.
- Low severity is easiest; impact level needs richer context in generation.

2. Service misclassification (rules)

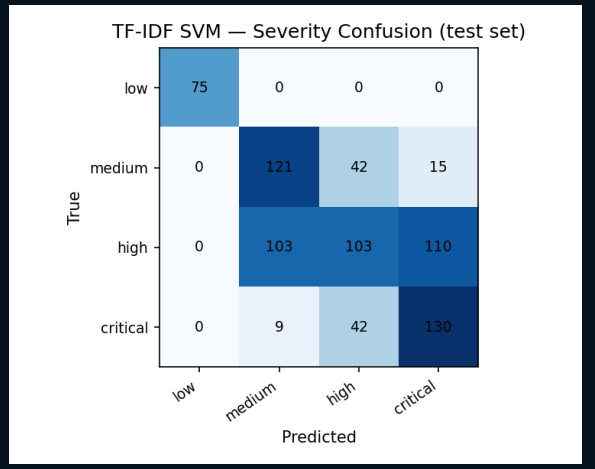
- Keyword rules reach ~0.80 cause F1 but only ~0.25 service F1.
- Rules count service mentions instead of identifying the failing component.

3. Lexical shortcut success (TF-IDF)

- Near-perfect cause/service/action F1 suggests strong keyword cues in synthetic logs.
- Motivates harder generation: paraphrases, misleading symptoms, weaker label leakage.

4. Evidence-line heuristic

- Shared heuristic reaches ~0.86–0.89 F1 across models.
- Future work: train a dedicated line-level evidence selector.



Conclusion

Did we achieve the desired result?

Yes: the project demonstrates a full NLP pipeline for explainable log triage. The package includes synthetic data, validation, EDA, baselines, metrics, visuals, report, prompts, and code. The results show that structured triage is feasible on controlled synthetic logs.

Main takeaway

LogTriage converts noisy logs into operator-ready decisions with evidence.

What we learned

Fixed taxonomies are essential; free-form actions are hard to evaluate.
Synthetic data must be validated because LLM labels may be inconsistent.
High model scores can indicate useful patterns, but also possible lexical shortcuts.
Evidence-line metrics are important for explainability and debugging.

Scope choice

E-commerce was chosen for clarity; the method can adapt to SaaS, banking, delivery, or cloud logs.

Future work

LLM zero/few-shot evaluated on n=30 with gpt-4o-mini; scale to full test set when budget allows.
DistilRoBERTa evaluated on n=75 test subset; scale to full data for stronger comparison.
Add harder examples with misleading noise and weaker keyword cues.
Manually review a gold subset and test on real anonymized logs.

Final repository artifacts

data/
src/
prompts/
results/
visuals/
slides/
docs/

Reproduce: python src/train_baselines.py • Report: docs/Model_Evaluation_Report.md